

OMAT

Ordres de Mission — Algérie Télécom

Documentation technique de l'application

Version 1.0 — livrable de fin de stage
Date 27 septembre 2026
Dépôt `omat-order-mission-algerie-telecom` (monorepo npm workspaces)
Pile NestJS 10 · Prisma 5 · PostgreSQL 17 · React 18 · Vite · Tailwind CSS 4 · Docker · nginx

Objet du document. Ce document décrit l'architecture, la conception et la mise en œuvre de l'application OMAT : découpage en composants, modèle de données, flux métier, sécurité, génération des documents, internationalisation, tests et déploiement. Il s'adresse aux équipes techniques chargées de la reprise, de l'exploitation ou de l'évolution du système.

Sommaire

1	Contexte et périmètre fonctionnel	2
2	Architecture générale	2
2.1	Organisation du dépôt	2
3	Architecture du backend (NestJS)	3
3.1	Traitement d'une requête	3
3.2	Principaux points d'entrée REST	3
4	Modèle de données	3
5	Flux métier	4
5.1	Cycle de vie d'un ordre de mission et de son décompte	4
5.2	Calcul du montant	5
6	Sécurité	5
6.1	Authentification et sessions	5
6.2	Autorisation : politique d'accès centralisée	6
6.3	Défense en profondeur	6
7	Architecture du frontend (React)	6
7.1	Internationalisation et bidirectionnalité	7
8	Génération des documents PDF	7
9	Qualité, tests et intégration continue	7
10	Déploiement et exploitation	7
11	Décisions d'architecture et perspectives	8

1 Contexte et périmètre fonctionnel

OMAT dématérialise le cycle de vie d'un déplacement professionnel d'un agent d'Algérie Télécom : de la demande d'**ordre de mission**, à sa **validation** par un administrateur, jusqu'au **décompte** des frais (repas, nuitées, kilométrage) et à sa décision (acceptation ou rejet motivé).

Terme métier	Définition
Ordre de mission	Demande de déplacement d'un agent ; un enregistrement par déplacement. États : <i>En cours</i> (INPROGRESS), <i>Validé</i> (COMPLETED).
Décompte	Règlement des frais rattaché à un ordre validé. États : <i>En attente</i> , <i>Accepté</i> , <i>Rejeté</i> .
Direction	Zone de la mission : Nord, Sud, ou « Nord et Sud » (MIXTE).
Barème	Taux par catégorie d'agent (repas, hébergement, indemnité kilométrique), différenciés par zone.
Exercice	Année fiscale qui délimite ordres et décomptes ; un exercice est toujours sélectionné.
Structure	Service/unité organisationnelle d'un agent ; délimite le périmètre d'un administrateur.

Acteurs et rôles

- **USER** (agent) : crée et suit ses propres ordres de mission et décomptes.
- **ADMIN** : valide les ordres, accepte/rejette les décomptes, gère les agents *de sa structure*.
- **SUPER_ADMIN** : périmètre global, barèmes, structures, réouverture des dossiers verrouillés, suppression définitive, journal d'audit.

2 Architecture générale

L'application suit une architecture **trois tiers** conteneurisée. Un reverse-proxy nginx termine le TLS, sert le client React (SPA statique) et relaie `/api` vers l'API NestJS. La base PostgreSQL est placée sur un réseau Docker *interne*, inaccessible depuis l'extérieur.

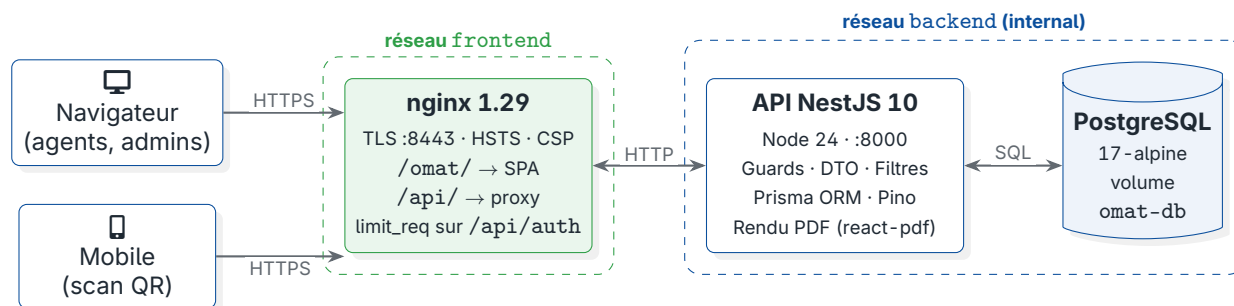


FIGURE 1 – Architecture de déploiement (Docker Compose).

2.1 Organisation du dépôt

Le code est un **monorepo npm workspaces** : une seule `package-lock.json`, des scripts racine (`npm run dev, build, test, lint`) qui orchestrent les deux paquets.

Répertoire	Rôle
<code>server/</code>	API NestJS (@omat/server) : modules métier, Prisma (<code>prisma/schema.prisma</code> , migrations, seeds), tests Jest unitaires et e2e.
<code>client/</code>	SPA React (@omat/client) : pages, composants UI, i18n <code>fr/ar</code> , tests Vitest et Playwright.
<code>deploy/</code>	Configuration nginx de production (TLS, en-têtes de sécurité, proxy).
<code>docs/</code>	ADR (<code>docs/adr/</code>), plan et rapport de sécurité, workflow des décomptes.
<code>.github/</code>	Intégration continue (<code>ci.yml</code>), analyse CodeQL, Dependabot.

3 Architecture du backend (NestJS)

Le serveur est une application NestJS modulaire. Chaque domaine métier est un module (contrôleur + service + DTO) ; les préoccupations transverses (authentification, autorisation, validation, journalisation, gestion d'erreurs) sont appliquées globalement.

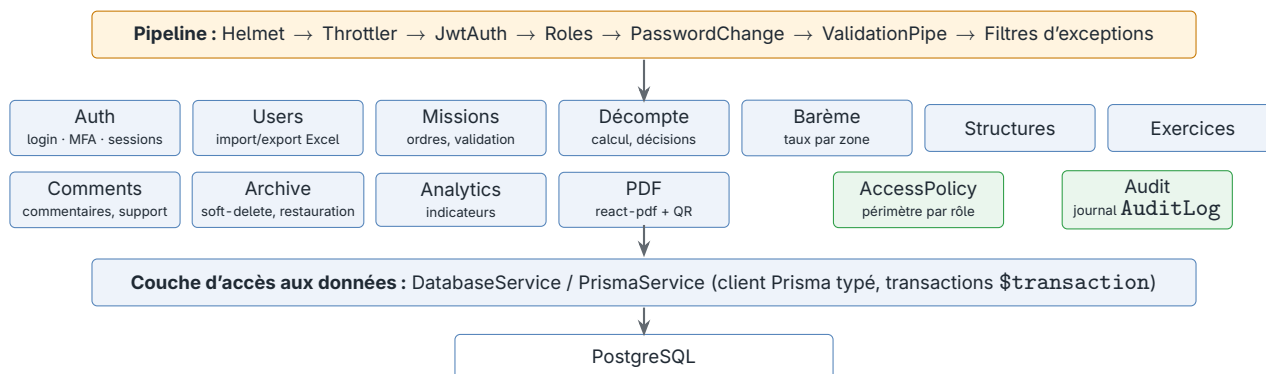


FIGURE 2 – Découpage modulaire de l'API et pipeline de traitement d'une requête.

3.1 Traitement d'une requête

- En-têtes et limites** : Helmet impose une CSP `default-src 'none'` (l'API ne sert que du JSON et des fichiers) ; corps JSON limité à 100 ko ; `trust proxy` pour récupérer l'IP réelle derrière nginx.
- Gardes globales** (ordre significatif) : limitation de débit (300 req/min/IP, 5/min sur les routes d'identification) ; vérification du JWT ; contrôle du rôle via le décorateur `@Roles` ; blocage de toute route tant qu'un mot de passe temporaire n'a pas été changé. Les routes publiques sont marquées `@Public()`.
- Validation** : `ValidationPipe` global avec `whitelist` et `forbidNonWhitelisted` ; seules des classes DTO (`class-validator`) sont acceptées en entrée, jamais des types Prisma — ce qui empêche l'assignation de masse.
- Service métier** : applique la politique d'accès, les règles métier et persiste via Prisma, en transaction lorsque plusieurs écritures doivent être atomiques.
- Filtres d'exceptions** : `PrismaClientExceptionFilter` (contraintes d'unicité, enregistrements absents) → `HttpExceptionFilter` → `AllExceptionsFilter` ; réponses d'erreur homogènes, sans fuite de pile d'appels.

3.2 Principaux points d'entrée REST

Ressource	Opérations
/auth	login, mfa/setup, mfa/verify, refresh, logout
/missions	CRUD, GET /user (mes ordres), GET /:id/download (PDF), POST /:id/reopen
/decompte	création, PATCH /:id/accept reject, bulk/accept reject, /:id/download, /:id/reopen
/users, /structures	CRUD, import (upload) et export Excel, archivage, réinitialisation du mot de passe
/barem, /exercices	gestion des taux ; exercice courant
/archive	liste, archivage/restauration unitaire et en masse, suppression définitive
/analytics, /audit, /comments	tableaux de bord, journal d'audit, commentaires et demandes de support

La documentation OpenAPI (Swagger) est générée automatiquement et exposée en développement uniquement.

4 Modèle de données

Le schéma est défini dans `server/prisma/schema.prisma` et versionné par migrations Prisma. La suppression est **logique** (`soft_delete`, `archivedAt`, `archivedBy`) sur toutes les entités métier afin de préserver l'historique.

Énumérations : `Role` (USER, ADMIN, SUPER_ADMIN), `Category` (CADRE, CADRE_SUPERIEUR, EXECUTION_MAITRISE), `Direction` (NORD, SUD, MIXTE), `TransportType` (véhicule de service, véhicule personnel, transport entreprise/employé), `MissionStatus`, `DecompteStatus`.

Choix de conception notables

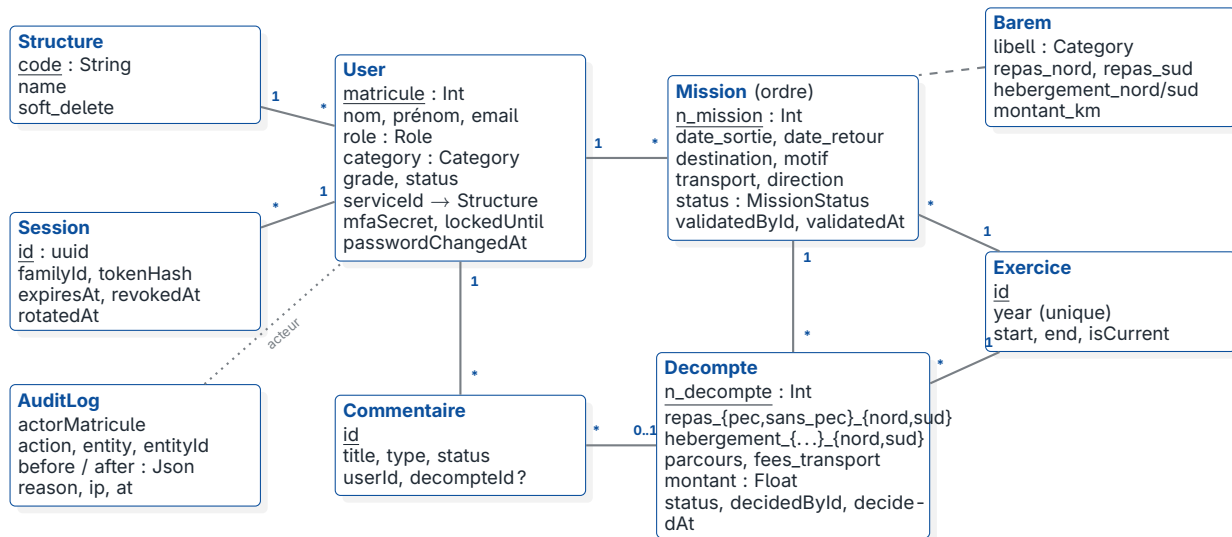


FIGURE 3 – Modèle conceptuel des données (entités principales et cardinalités).

- Le *verrou* d'un dossier est **dérivé du statut** (ordre validé, décompte accepté/rejeté) : pas de colonne `lockedAt` redondante susceptible de diverger.
- Les compteurs du décompte sont **ventilés par zone** (8 colonnes) pour payer chaque repas et nuitée au barème de la zone où il a été consommé (ADR 0003).
- `exerciceId` est dénormalisé sur le décompte pour simplifier le filtrage par année fiscale.
- Les jetons de rafraîchissement ne sont jamais stockés en clair : seul un HMAC-SHA256 est conservé.

5 Flux métier

5.1 Cycle de vie d'un ordre de mission et de son décompte

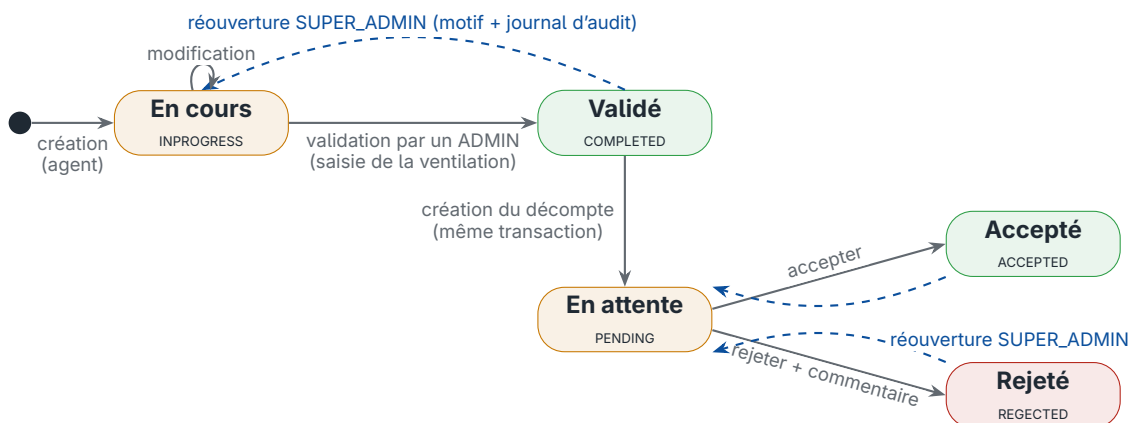


FIGURE 4 – Machine à états : ordre de mission (haut) et décompte (bas).

Règles appliquées côté serveur

- **Pas d'auto-approbation** : un ADMIN ne peut ni valider son propre ordre, ni décider de son propre décompte. L'auteur et l'horodatage de la décision sont enregistrés (`validatedById`, `decidedById`).
- **Cohérence des quantités** : le nombre de repas et de nuitées déclarés est vérifié par rapport aux dates de sortie et de retour ; une ventilation hors de la Direction de l'ordre est refusée.
- **Rejet motivé** : tout rejet crée un `Commentaire` visible par l'agent.
- **Actions en masse** : acceptation/rejet et archivage de plusieurs dossiers en une requête, chacun contrôlé par la politique d'accès.

5.2 Calcul du montant

Une fonction pure unique, `computeMontant (server/src/decompte/montant.ts)`, est partagée par la création, la mise à jour et le jeu de données de démonstration. Elle est donc testable isolément et constitue le seul point à modifier si la règle de gestion évolue.

```
let montant =
  (nord.heb_pec + nord.heb_sans_pec) * b.hebergement_nord
+ (nord.repas_pec + nord.repas_sans_pec) * b.repas_nord
+ sud.heb_sans_pec * b.hebergement_sud
+ sud.repas_sans_pec * b.repas_sud;
if (transport === PERSONAL_CAR)
  montant += parcours * b.montant_km;
if (anyPec) montant *= 0.25;
return montant + fees_transport;
```

- Chaque zone est valorisée à **son propre barème** (selon la catégorie de l'agent).
- Nord : tous les repas et nuitées sont payés ; Sud : seuls les éléments *sans prise en charge*.
- Toute prise en charge (*pec*) ramène le total à 25 %.
- Les frais de transport sont ajoutés en dernier.

6 Sécurité

La sécurité a fait l'objet d'un programme de durcissement dédié, documenté par deux ADR (`docs/adr/0001`, `0002`), un plan de sécurité et un rapport de test d'intrusion (`docs/security/`). La menace principale retenue est l'**utilisateur interne** dépassant son rôle.

6.1 Authentification et sessions

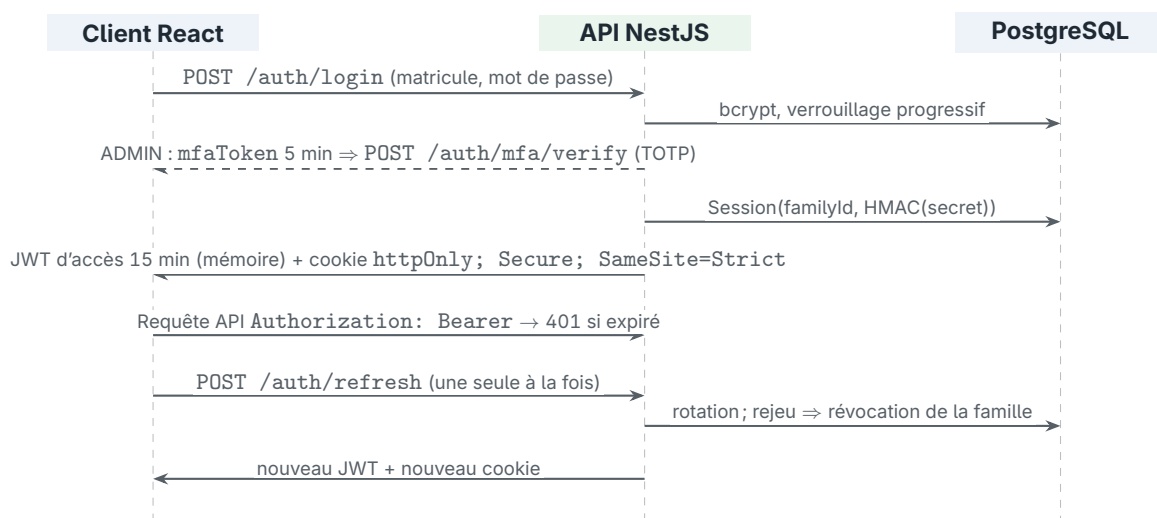


FIGURE 5 – Séquence d'authentification, MFA et rafraîchissement de session.

- **Jeton d'accès** : JWT HS256 de 15 minutes (*iss/aud* contrôlés), conservé *uniquement en mémoire* côté client — inaccessible à un script injecté via le stockage local. À chaque requête, le rôle et le statut sont relus en base et les jetons émis avant `passwordChangedAt` sont refusés.
- **Jeton de rafraîchissement** : valeur opaque `<id>.<secret>`, seul un HMAC est stocké. La rotation crée une nouvelle session dans la même famille ; la réutilisation d'un jeton déjà tourné révoque toute la famille (détection de vol), avec une tolérance de 30 s pour les onglets concurrents. Le client sérialise les rafraîchissements (*single-flight*).
- **MFA TOTP** obligatoire pour ADMIN et SUPER_ADMIN : secrets chiffrés AES-256-GCM, un code accepté une seule fois par pas de 30 s, dix codes de secours hachés (SHA-256).
- **Mots de passe** : hachage bcrypt ; mot de passe temporaire imposé à la création ou après réinitialisation ; verrouillage exponentiel (1, 2, 4... jusqu'à 60 min) à partir du 5^e échec.
- **CSRF** : les appels API ne portent pas de cookie ; `/auth/refresh` et `/auth/logout` vérifient en plus l'en-tête `Origin`.

6.2 Autorisation : politique d'accès centralisée

La classe `AccessPolicy` (`server/src/common/policy/`) est la **source unique de vérité** sur « qui voit et modifie quoi ». Elle produit des fragments `where` Prisma fusionnés dans chaque requête :

```
scopeMissions(actor) {
  if (isSuper(actor)) return {}; // tout
  if (isAdmin(actor)) return { user: { serviceId: actor.serviceId } }; // sa structure
  return { userId: actor.matricule }; // ses propres ordres
}
// Service : db.mission.findFirst({ where: { n_mission: id, ...policy.scopeMissions(actor) } })
```

Une lecture hors périmètre renvoie **404** (les identifiants ne furent pas) ; une écriture visible mais interdite (auto-approbation, dossier verrouillé) renvoie **403**. Un test *e2e matrice d'autorisation* (toutes les routes × tous les rôles/périmètres) détecte tout nouveau point d'entrée qui omettrait la politique.

6.3 Défense en profondeur

Couche	Mesures
Réseau	TLS, HSTS; base sur réseau Docker internal; conteneur API non-root.
nginx	CSP stricte (<code>script-src 'self'</code>), <code>X-Content-Type-Options</code> , <code>Referrer-Policy</code> , <code>Permissions-Policy</code> ; limitation de débit sur <code>/api/auth</code> .
API	Helmet, limitation de débit applicative, validation stricte des DTO, taille des corps bornée, validation du contenu des fichiers importés.
Données	Traçabilité complète dans <code>AuditLog</code> (acteur, avant/après, motif, IP) : décisions, réouvertures, suppressions définitives.
Chaîne logicielle	CodeQL, Dependabot, <code>npm audit -audit-level=high</code> en CI.

7 Architecture du frontend (React)

Le client est une SPA React 18 + TypeScript construite avec Vite et servie sous le préfixe `/omat/`.

React Router 6 — `ProtectedRoute` (session) · `RedirectBasedRole` (tableau de bord selon le rôle) · `/scan/:type/:id` (QR)

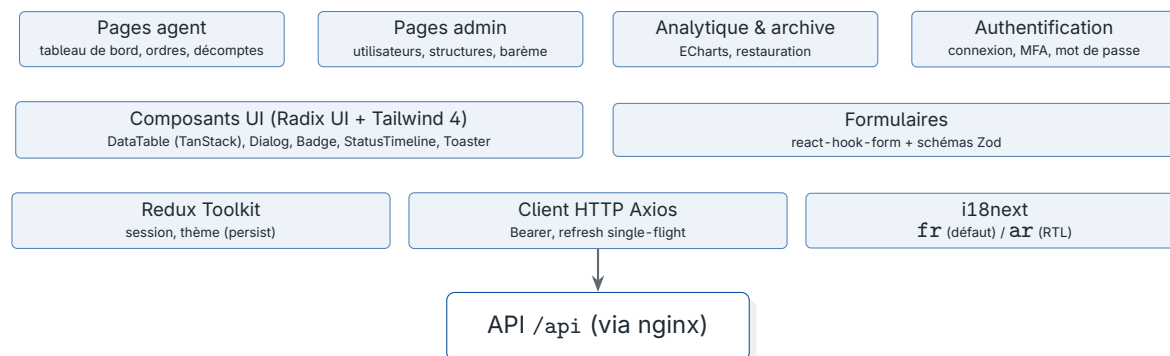


FIGURE 6 – Organisation en couches du client React.

- **Gestion de session** : le jeton d'accès n'est jamais persisté ; au rechargement de la page, la session est restaurée via `/auth/refresh`. L'intercepteur Axios rejoue les requêtes en 401 après un rafraîchissement unique partagé entre les appels concurrents.
- **Design system** : primitives accessibles (Radix UI) stylées avec Tailwind CSS 4 ; typographie IBM Plex Sans / Plex Sans Arabic ; thème clair/sombre.
- **Tableaux** : TanStack Table pour tri, filtrage, pagination et sélection multiple (actions en masse).
- **Visualisation** : tableau de bord analytique ECharts (montants et volumes par Direction, par statut, par exercice), restreint à la structure de l'ADMIN.
- **Exports** : téléchargement des PDF et exports Excel (délai réseau porté à 60 s pour les fichiers).

7.1 Internationalisation et bidirectionnalité

L'interface est disponible en **français** (défaut) et en **arabe** (droite-à-gauche), sélectionnable dans l'en-tête et mémorisée par navigateur.

- Toute chaîne visible provient de `client/src/locales/<fr|ar>/<namespace>.json`; un test (`locales.test.ts`) garantit la parité des clés et une règle ESLint (`eslint-plugin-i18next`) interdit le texte brut dans le JSX.
- La mise en page n'utilise que des propriétés *logiques* (`ms-/me-`, `start-/end-`); les icônes directionnelles sont retournées en RTL.
- Les nombres conservent les chiffres latins et le format français (1 500,50 DA en français, le symbole arabe du dinar en arabe).

8 Génération des documents PDF

Les ordres de mission et décomptes sont produits **côté serveur** avec `@react-pdf/renderer` : les documents sont décrits comme des composants React (`server/src/pdf/documents/*.tsx`), ce qui permet de réutiliser des primitives de mise en page (en-tête, logo vectoriel, tableaux) et de tester le rendu.

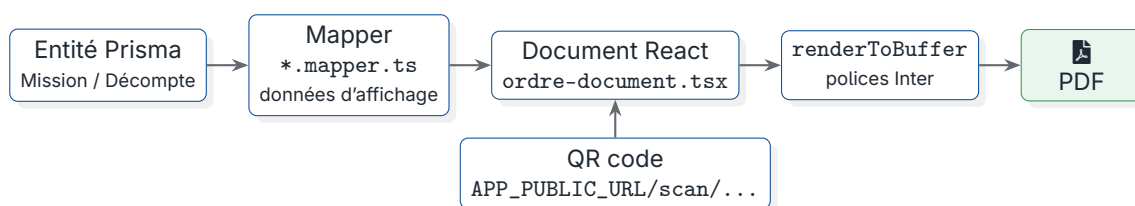


Figure 7 – Chaîne de génération d'un document PDF.

Chaque document porte un **QR code** pointant vers la route `/scan/<ordre|decompte>/<id>` du client : scanné depuis un téléphone, il redirige, après authentification, vers la page de détail correspondante, dans la limite des droits de l'utilisateur. Les *mappers* isolent la transformation des données (formatage des dates, montants, ventilation Nord/Sud) du rendu, et sont testés unitairement.

9 Qualité, tests et intégration continue

Niveau	Outil	Portée
Unitaire serveur	Jest	services (missions, décomptes, barème, archive...), calcul du montant, mappers PDF, QR, filtres d'erreurs, utilitaires cryptographiques.
E2E serveur	Jest + Supertest	base PostgreSQL réelle : authentification, sessions (rotation, rejeu), matrice de contrôle d'accès, métadonnées des routes (<code>@Public</code> , <code>@Roles</code>).
Unitaire client	Vitest + Testing Library	composants UI, client HTTP, parité des traductions, redirection de scan.
E2E client	Playwright + axe-core	parcours complets (ordres, décomptes, archive, analytique, i18n, session) et audit d'accessibilité.

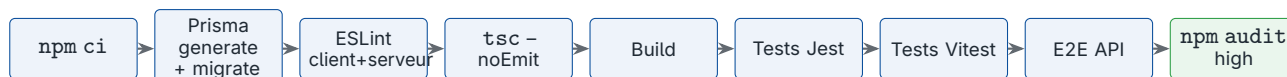


Figure 8 – Pipeline GitHub Actions (service PostgreSQL 17 éphémère, Node 24).

10 Déploiement et exploitation

- **Images Docker multi-étapes** : l'étape de build installe les dépendances du seul workspace concerné, génère le client Prisma et compile; l'image d'exécution ne contient que les dépendances de production et s'exécute sous un utilisateur non privilégié `omat`. Le client est compilé puis copié dans une image

nginx:1.29-alpine.

- **Démarrage** : `docker compose up -d`; le point d'entrée du serveur applique les migrations (`prisma migrate deploy`) avant le lancement; l'API attend que la base soit *healthy*.
- **Configuration** : variables validées au démarrage (`validateEnv`) — `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `MFA_ENCRYPTION_KEY`, `APP_PUBLIC_URL`, `POSTGRES_PASSWORD`; certificats TLS montés depuis `deploy/certs`. Aucun secret n'est versionné.
- **Journalisation** : logs structurés JSON (Pino) avec IP client réelle, exploitables par un collecteur centralisé.
- **Données initiales** : `npm run seed:db` (référentiel, barèmes, comptes initiaux) et `npm run seed:demo` (jeu de démonstration).

Démarrage local rapide. `npm install` → copier `server/.env.example` vers `server/.env` → `npm run setup:db` → `npm run dev` (API sur `:8000`, client Vite avec proxy `/api`).

11 Décisions d'architecture et perspectives

ADR	Décision
0001	Contrôle d'accès centralisé au niveau des requêtes (<code>AccessPolicy</code>), 404 hors périmètre, pas d'auto-approbation, verrouillage des dossiers approuvés.
0002	Jeton d'accès court en mémoire, sessions de rafraîchissement opaques, hachées et révocables, MFA pour les administrateurs.
0003	Ordres « Nord et Sud » : ventilation des repas et nuitées par zone, chacune valorisée à son barème.

Perspectives

- Brancher l'authentification sur le SSO de l'entreprise lorsqu'il sera disponible (remplacement des mots de passe locaux et du TOTP).
- Faire valider par la direction métier la règle d'abattement de 25 % et le traitement des éléments pris en charge au Sud; seule `computeMontant` serait à modifier.
- Stocker le montant par zone si un reporting financier ventilé Nord/Sud est requis.
- Ajouter des notifications (courriel) lors des décisions sur les décomptes.